Инженерия данных

Лабораторная работа 3. Технология MapReduce

Выполнила: Гилязова Алина Айратовна, 465533, J3312
Индивидуальный вариант № 2: **«Программист, разработчик на Python»**.

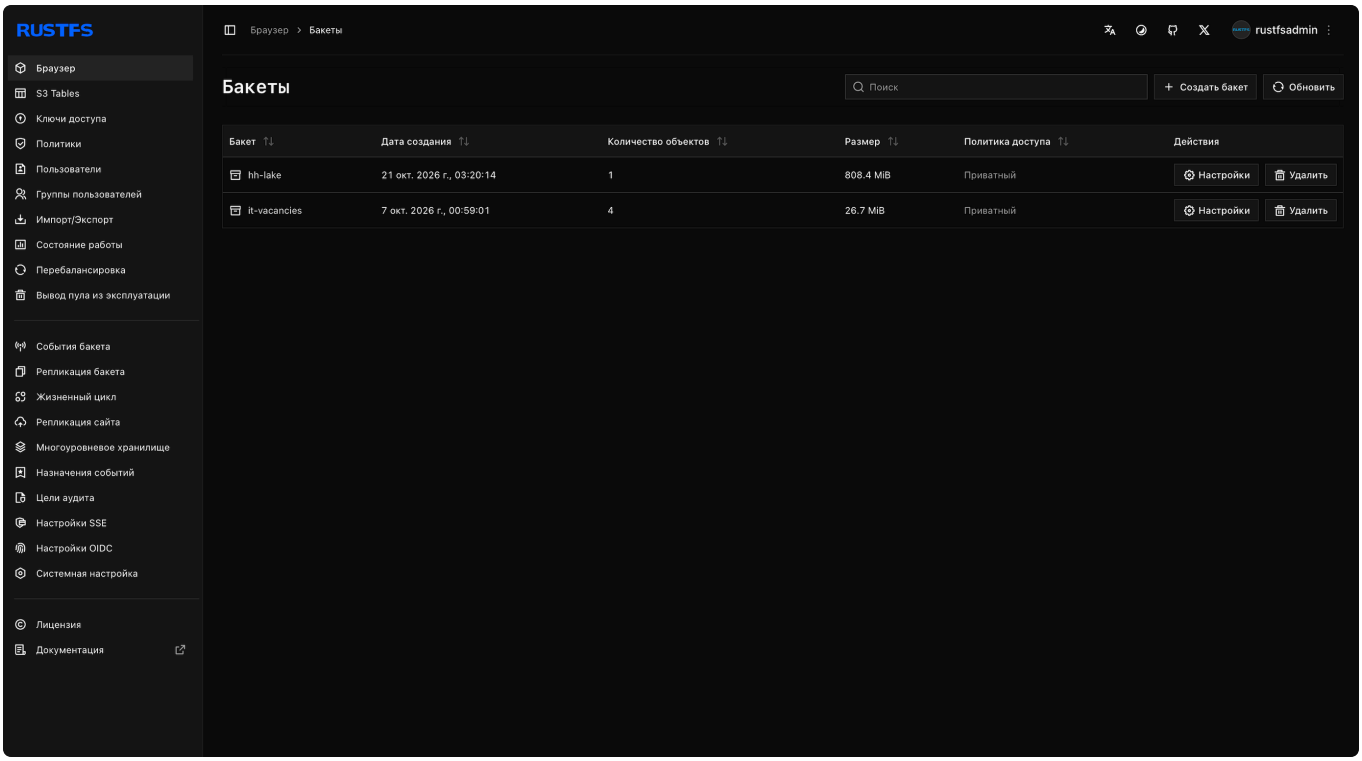
1. Требования

Необходимо развернуть RustFS, загрузить датасет IT-вакансий и рассчитать частоты навыков, пар и троек двумя способами: собственной реализацией MapReduce и SQL-запросами DuckDB. После этого нужно проверить совпадение результатов, проанализировать медианные зарплаты, сравнить три региона и измерить время выполнения и пиковую память. Исходный код и PDF-отчёт должны быть опубликованы на GitHub.

2. Объектное хранилище и исходные данные

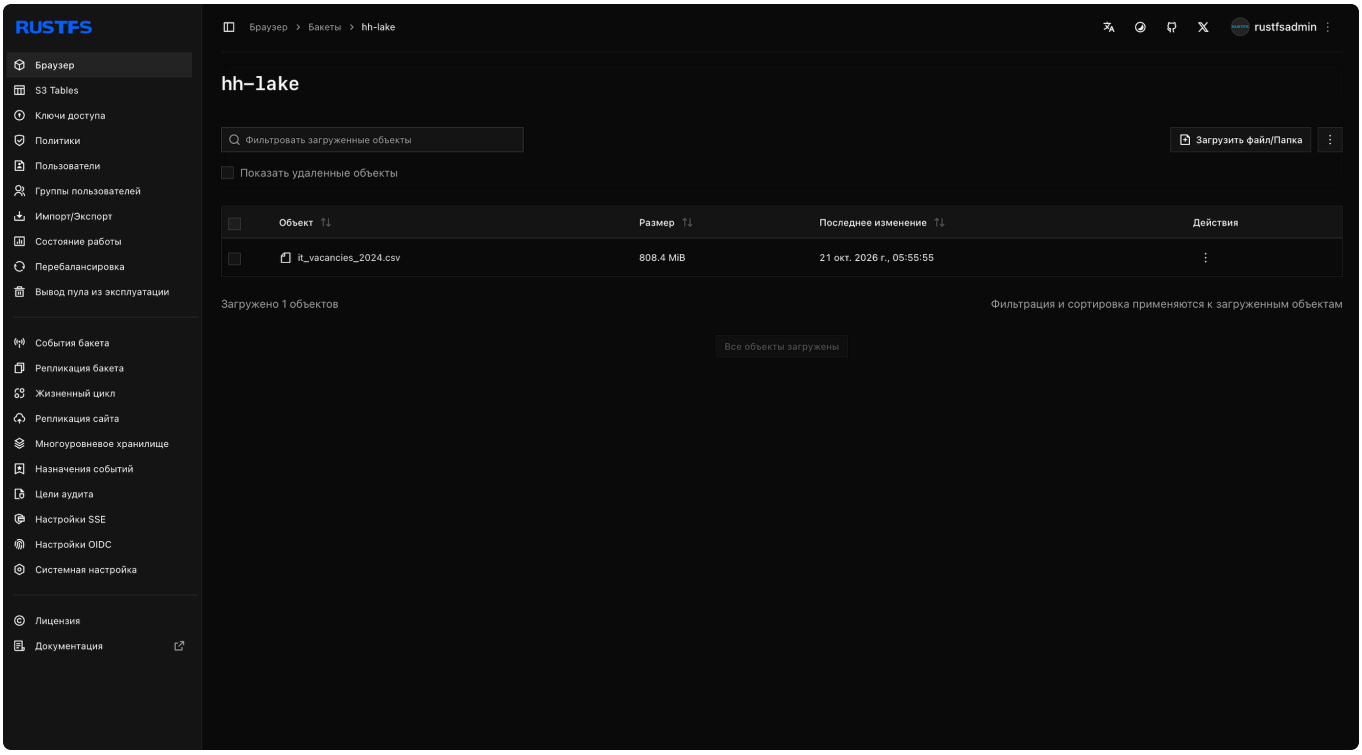
Для работы использовала локальное хранилище RustFS с четырьмя папками-дисками `disk1` – `disk4` и конфигурацией `EC:2`, оставшимися со второй лабораторной. Адрес S3 API — `127.0.0.1:9000`, веб-консоли — `127.0.0.1:9001`. Создала бакет `hh-lake`.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ python check_s3.py
[Введите секретный ключ RustFS:
 Подключение к RustFS успешно!
 Файл найден: it_vacancies_2024.csv
 Размер файла: 847619384 байт
```



Загрузила предоставленный преподавателем файл `it_vacancies_2024.csv` (847 619 384 байта, около 808,35 MiB). В датасете содержатся названия вакансий, роли, навыки, зарплатные границы, валюта и регион.

```
~/rustfs/bin/rc ls lab/hh-lake/
```



Подключение Python к объектному хранилищу проверила через `boto3` и операцию `head_object`. Файл найден без предварительного скачивания из S3.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ ls -ld ~/rustfs/disks/disk{1..4}
drwxr-xr-x  5 macbook  staff  160 21 окт.  03:20 /Users/macbook/rustfs/disks/disk1
drwxr-xr-x  5 macbook  staff  160 21 окт.  03:20 /Users/macbook/rustfs/disks/disk2
drwxr-xr-x  5 macbook  staff  160 21 окт.  03:20 /Users/macbook/rustfs/disks/disk3
drwxr-xr-x  5 macbook  staff  160 21 окт.  03:20 /Users/macbook/rustfs/disks/disk4
```

3. Подсчёт навыков через MapReduce

В исходном датасете **229 388 вакансий**. Для своего варианта оставила записи, у которых `role = 'Программист, разработчик'`, а в названии есть `python` независимо от регистра. Получилось **3 894 вакансии**. Из них у **2 986** заполнен список `key_skills`, у **908** список пустой.

```
role == "Программист, разработчик" and "python" in name.lower()
```

Такое условие исключает другие профессиональные роли, например тестировщиков. При этом оно не включает разработчиков Python, у которых язык указан только в навыках, но отсутствует в названии. Это выбранное правило отбора, одинаковое для обоих методов.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ python check_filter.py
Другие языки в названиях:
go 389
c++ 170
php 102
java 88
golang 86
c# 61
1c 35
1c 2

Примеры вакансий:
Junior Python / Go Developer (Microservices)
Специалист Python \ Java направления «Автоматизация процессов интегрированного планирования»
Python (Django) программист Middle
Python/Go-разработчик в команду Клиентских и внутренних сервисов
Программист C/C++ / Реверс-инженер
Разработчик интеграций по работе с маркетплейсами (Python/PHP)
Ведущий разработчик C/C++
Senior developer C++/Python
Backend-разработчик Python Django
Ведущий программист 1С ERP
Middle/Senior C++ программист
Python разработчик (django) middle
Algorithms and Models Engineer (AME)
Algorithms and Models Engineer for L2+ layer (4G/LTE, 5G)
Golang Developer (Уфа)
Golang Developer (Уфа)
Разработчик Django/ReactJS
Руководитель разработки Full Stack (Java/React)
Junior C/C++ разработчик (Embedded Linux)
Разработчик Django/ReactJS
```

Собственная программа `mapreduce.py` читает объект из RustFS через `s3.get_object()` и обрабатывает записи последовательно, чанками по **1 000 вакансий**. Всего обработано

230 чанков. Проверку `role` и `name` применяла при обработке каждого чанка.

```
response = s3.get_object(Bucket="hh-lake", Key="it_vacancies_2024.csv")
reader = csv.DictReader(
    io.TextIOWrapper(response["Body"], encoding="utf-8-sig", newline="")
)

# Map: одна вакансия — комбинации навыков
skills = sorted({s.strip() for s in (row["key_skills"] or "").split(";") if
s.strip()})
for size in (1, 2, 3):
    partial[size].update(combinations(skills, size))

# Reduce: объединение частичных счётчиков
for size in (1, 2, 3):
    totals[size].update(partial[size])
```

Основной цикл собирает очередные 1 000 записей и передаёт их мапперу. Последний неполный чанк обрабатывается отдельно. В программе подсчитываются только данные, прошедшие фильтр варианта.

На стадии **Map** для каждого чанка создаются счётчики комбинаций. На стадии **Shuffle/Reduce** комбинации с одинаковыми ключами объединяются, а частоты суммируются. Повторы одного навыка внутри вакансии исключаются через `set`.

Размер набора	Уникальных комбинаций	Самая частая комбинация	Частота
1 навык	1 426	Python	2 857
2 навыка	19 259	PostgreSQL + Python	1 306
3 навыка	120 466	Git + PostgreSQL + Python	475

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
(.venv) > $ python mapreduce.py
Введите секретный ключ RustFS:

РЕЗУЛЬТАТЫ MAPREDUCE
Всего вакансий: 229388
Python-вакансий: 3894
Python-вакансий с навыками: 2986
Обработано чанков: 230

Комбинации из 1 навыков:
Уникальных комбинаций: 1426
Python 2857
PostgreSQL 1347
Git 943
Docker 821
SQL 792
Django Framework 743
Linux 689
Redis 485
FastAPI 474
Django 366
Сохранено: s3://hh-lake/results/skills_1.csv

Комбинации из 2 навыков:
Уникальных комбинаций: 19259
PostgreSQL + Python 1306
Git + Python 905
Docker + Python 779
Python + SQL 775
Django Framework + Python 722
Linux + Python 662
Git + PostgreSQL 494
Docker + PostgreSQL 479
Python + Redis 457
FastAPI + Python 450
Сохранено: s3://hh-lake/results/skills_2.csv

Комбинации из 3 навыков:
Уникальных комбинаций: 120466
```

```
Git + PostgreSQL + Python 475
Docker + PostgreSQL + Python 459
Django Framework + PostgreSQL + Python 429
PostgreSQL + Python + SQL 364
Git + Linux + Python 344
Docker + Git + Python 333
Git + Python + SQL 299
PostgreSQL + Python + Redis 294
Django Framework + Git + Python 280
Docker + Python + SQL 270
Сохранено: s3://hh-lake/results/skills_3.csv
```

При увеличении размера набора число различных комбинаций растёт. Если в вакансии есть `n` разных навыков, она даёт `n` одиночных навыков, `n(n-1)/2` пар и `n(n-1)(n-2)/6` троек. Например, при пяти навыках получается 5 одиночных комбинаций, 10 пар и 10 троек. Поэтому при обработке многих вакансий пар и троек может оказаться значительно больше, чем отдельных навыков.

Результаты сохранила непосредственно в RustFS:

```
s3://hh-lake/results/skills_1.csv
s3://hh-lake/results/skills_2.csv
s3://hh-lake/results/skills_3.csv
```

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ ~/rustfs/bin/rc ls lab/hh-lake/results/
 [2026-10-21 03:18:27]  27.18 KiB results/skills_1.csv
 [2026-10-21 03:18:28] 556.62 KiB results/skills_2.csv
 [2026-10-21 03:18:28]   4.57 MiB results/skills_3.csv
```

4. Независимая проверка через DuckDB

Второй способ использует DuckDB и расширение `httpfs`, которое читает исходный CSV по пути `s3://hh-lake/it_vacancies_2024.csv`. Условие для варианта написала отдельно в SQL:

```
WHERE role = 'Программист, разработчик'
  AND contains(lower(coalesce(name, '')), 'python')
```

Список навыков раскрывается через `UNNEST(string_split(key_skills, ';'))`, затем удаляются повторы внутри вакансии. Основные операции SQL:

```sql
-- Одиночные навыки
SELECT a.skill, COUNT(*) AS frequency
FROM skills a
GROUP BY a.skill;

-- Пары навыков
SELECT a.skill, b.skill, COUNT(*) AS frequency
FROM skills a
JOIN skills b ON a.vacancy_num = b.vacancy_num
             AND a.skill < b.skill
GROUP BY a.skill, b.skill;

-- Тройки навыков
SELECT a.skill, b.skill, c.skill, COUNT(*) AS frequency
FROM skills a
JOIN skills b ON a.vacancy_num = b.vacancy_num
             AND a.skill < b.skill
JOIN skills c ON b.vacancy_num = c.vacancy_num
             AND b.skill < c.skill
GROUP BY a.skill, b.skill, c.skill;
```

Здесь `skills` — подготовленный набор уникальных навыков каждой вакансии, полученный из исходного CSV внутри CTE. Сравнение названий через `<` исключает повторный подсчёт сочетаний в другом порядке.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ python duckdb_check.py
[Введите секретный ключ RustFS:
 Всего вакансий через DuckDB: 229388
```

Размер набора	MapReduce	DuckDB	Проверка частот
1 навык	1 426	1 426	Совпадают полностью
2 навыка	19 259	19 259	Совпадают полностью
3 навыка	120 466	120 466	Совпадают полностью

Программа `duckdb_skills.py` сравнила словари «комбинация — частота» для всех трёх размеров. Расхождений не обнаружено. Таким образом, совпали не только количества комбинаций, но и частоты каждой из них.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ python duckdb_skills.py
[Введите секретный ключ RustFS:

Проверка комбинаций из 1 навыков
DuckDB: 1426
MapReduce: 1426
Результаты полностью совпадают!

Проверка комбинаций из 2 навыков
DuckDB: 19259
MapReduce: 19259
Результаты полностью совпадают!

Проверка комбинаций из 3 навыков
DuckDB: 120466
MapReduce: 120466
Результаты полностью совпадают!
```

5. Анализ влияния сочетаний навыков на зарплату

Среди **3 894** Python-вакансий зарплата указана у **1 446**. Для сопоставимости использовала только **1 332** вакансии с зарплатой в рублях (`RUR`), без пересчёта USD, EUR и KZT. Из них у **1 095** указаны навыки.

Если есть обе границы зарплаты, брала середину диапазона; при наличии одной границы использовала её. Для каждого набора навыков рассчитала медиану. Чтобы не делать выводы по единичным объявлениям, оставила комбинации минимум с **5 вакансиями**, имеющими зарплату и навыки.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
(.venv) > $ python check_salary.py
Всего Python-вакансий: 3894
С указанной зарплатой: 1446
Без зарплаты: 2448

Валюты:
RUR 1332
USD 98
EUR 15
KZT 1
```

Размер набора	Комбинаций с ≥5 зарплатами	Пример набора	Медианная зарплата
1 навык	160	Python	156 250 ₽
2 навыка	1 032	PostgreSQL + Python	175 000 ₽
3 навыка	2 665	Git + PostgreSQL + Python	170 000 ₽

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
(.venv) > $ python salary_medians.py
Python-вакансий с зарплатой в рублях: 1332
Из них с заполненными навыками: 1095

Комбинации из 1 навыков
Комбинаций с минимум 5 вакансиями: 160
Топ-10 по количеству вакансий:
Python | вакансий: 1048 | медиана: 156250.0 руб.
PostgreSQL | вакансий: 495 | медиана: 175000.0 руб.
Git | вакансий: 462 | медиана: 155000.0 руб.
SQL | вакансий: 351 | медиана: 170000.0 руб.
Docker | вакансий: 301 | медиана: 180000.0 руб.
Django Framework | вакансий: 270 | медиана: 147500.0 руб.
Linux | вакансий: 252 | медиана: 150000.0 руб.
Redis | вакансий: 246 | медиана: 200000.0 руб.
FastAPI | вакансий: 177 | медиана: 175000.0 руб.
Django | вакансий: 156 | медиана: 195000.0 руб.
Сохранено: salary_medians_1.csv

Комбинации из 2 навыков
Комбинаций с минимум 5 вакансиями: 1032
Топ-10 по количеству вакансий:
PostgreSQL + Python | вакансий: 473 | медиана: 175000.0 руб.
Git + Python | вакансий: 445 | медиана: 155000.0 руб.
Python + SQL | вакансий: 342 | медиана: 170000.0 руб.
Docker + Python | вакансий: 291 | медиана: 180000.0 руб.
Django Framework + Python | вакансий: 262 | медиана: 145000.0 руб.
Linux + Python | вакансий: 246 | медиана: 150000.0 руб.
Git + PostgreSQL | вакансий: 235 | медиана: 170000.0 руб.
Python + Redis | вакансий: 227 | медиана: 200000.0 руб.
Docker + PostgreSQL | вакансий: 194 | медиана: 200000.0 руб.
PostgreSQL + SQL | вакансий: 186 | медиана: 175000.0 руб.
Сохранено: salary_medians_2.csv

Комбинации из 3 навыков
Комбинаций с минимум 5 вакансиями: 2665
Топ-10 по количеству вакансий:
Git + PostgreSQL + Python | вакансий: 223 | медиана: 170000.0 руб.
Docker + PostgreSQL + Python | вакансий: 186 | медиана: 200000.0 руб.
PostgreSQL + Python + SQL | вакансий: 181 | медиана: 175000.0 руб.
Docker + Git + Python | вакансий: 161 | медиана: 175000.0 руб.
Git + Python + SQL | вакансий: 156 | медиана: 172500.0 руб.
Git + Linux + Python | вакансий: 154 | медиана: 140000.0 руб.
Django Framework + PostgreSQL + Python | вакансий: 137 | медиана: 145000.0 руб.
Django Framework + Git + Python | вакансий: 136 | медиана: 125000.0 руб.
Git + Python + Redis | вакансий: 128 | медиана: 200000.0 руб.
Docker + Git + PostgreSQL | вакансий: 123 | медиана: 200000.0 руб.
Сохранено: salary_medians_3.csv
```

Для пар сравнивала их медиану с медианой **более частого одиночного навыка**. Для троек за основу брала **более частую пару** из данной тройки. Частоты базовых наборов оценивались среди вакансий, для которых можно посчитать зарплату. Разница рассчитывалась как медиана расширенного набора минус медиана базового.

Набор навыков	Базовый набор	Число вакансий с набором	Медиана базы	Медиана набора	Разница
AIOHTTP + Python	Python	8	156 250 ₽	420 000 ₽	**+263 750 ₽**
Cassandra + Python	Python	6	156 250 ₽	420 000 ₽	**+263 750 ₽**
JavaScript + Redis	Redis	6	200 000 ₽	50 000 ₽	**−150 000 ₽**
Qt + REST	REST	6	200 000 ₽	60 000 ₽	**−140 000 ₽**
AIOHTTP + PostgreSQL + Python	PostgreSQL + Python	8	175 000 ₽	420 000 ₽	**+245 000 ₽**
Cassandra + PostgreSQL + Python	PostgreSQL + Python	6	175 000 ₽	420 000 ₽	**+245 000 ₽**
Docker + JavaScript + REST API	Docker + REST API	8	250 000 ₽	70 000 ₽	**−180 000 ₽**
API + Git + Nginx	Git + Nginx	6	250 000 ₽	75 000 ₽	**−175 000 ₽**

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3                                  [6:29:57]
(.venv) > $ python salary_impact.py                                             ○ 24.14.1
[
 ПАРЫ НАВЫКОВ

[ПОВЫШАЮТ МЕДИАНУ:
 AIOHTTP + Python | база: Python | вакансий: 8 | базовая медиана: 156250 | медиана: 420000 | изменение: 263750
 Cassandra + Python | база: Python | вакансий: 6 | базовая медиана: 156250 | медиана: 420000 | изменение: 263750
 Elasticsearch + FastAPI | база: FastAPI | вакансий: 9 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 AIOHTTP + FastAPI | база: FastAPI | вакансий: 8 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 AIOHTTP + PostgreSQL | база: PostgreSQL | вакансий: 8 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 Asyncio + FastAPI | база: FastAPI | вакансий: 8 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 Cassandra + FastAPI | база: FastAPI | вакансий: 6 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 Cassandra + PostgreSQL | база: PostgreSQL | вакансий: 6 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
 Asyncio + Python | база: Python | вакансий: 19 | базовая медиана: 156250 | медиана: 400000 | изменение: 243750
 Asyncio + PostgreSQL | база: PostgreSQL | вакансий: 17 | базовая медиана: 175000 | медиана: 400000 | изменение: 225000

 ПОНИЖАЮТ МЕДИАНУ:
 JavaScript + Redis | база: Redis | вакансий: 6 | базовая медиана: 200000 | медиана: 50000 | изменение: −150000
 Qt + REST | база: REST | вакансий: 6 | базовая медиана: 200000 | медиана: 60000 | изменение: −140000
 REST + Unix | база: REST | вакансий: 6 | базовая медиана: 200000 | медиана: 60000 | изменение: −140000
 CSS + FastAPI | база: FastAPI | вакансий: 5 | базовая медиана: 175000 | медиана: 35000 | изменение: −140000
 FastAPI + HTML | база: FastAPI | вакансий: 5 | базовая медиана: 175000 | медиана: 35000 | изменение: −140000
 CSS + REST | база: REST | вакансий: 6 | базовая медиана: 200000 | медиана: 65000 | изменение: −135000
 PostgreSQL + Оптимизация кода | база: PostgreSQL | вакансий: 5 | базовая медиана: 175000 | медиана: 45000 | изменение: −130000
 Django + Веб-программирование | база: Django | вакансий: 5 | базовая медиана: 195000 | медиана: 67500 | изменение: −127500
 HTML + Nginx | база: Nginx | вакансий: 5 | базовая медиана: 206250 | медиана: 80000 | изменение: −126250
 HTML + REST | база: REST | вакансий: 8 | базовая медиана: 200000 | медиана: 75000 | изменение: −125000
```

```
ТРОЙКИ НАВЫКОВ

ПОВЫШАЮТ МЕДИАНУ:
AIOHTTP + PostgreSQL + Python | база: PostgreSQL + Python | вакансий: 8 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
Cassandra + PostgreSQL + Python | база: PostgreSQL + Python | вакансий: 6 | базовая медиана: 175000 | медиана: 420000 | изменение: 245000
AIOHTTP + FastAPI + Python | база: FastAPI + Python | вакансий: 8 | базовая медиана: 177500 | медиана: 420000 | изменение: 242500
Asyncio + FastAPI + Python | база: FastAPI + Python | вакансий: 8 | базовая медиана: 177500 | медиана: 420000 | изменение: 242500
Cassandra + FastAPI + Python | база: FastAPI + Python | вакансий: 6 | базовая медиана: 177500 | медиана: 420000 | изменение: 242500
Elasticsearch + FastAPI + Python | база: FastAPI + Python | вакансий: 6 | базовая медиана: 177500 | медиана: 420000 | изменение: 242500
Elasticsearch + FastAPI + PostgreSQL | база: FastAPI + PostgreSQL | вакансий: 9 | базовая медиана: 180000 | медиана: 420000 | изменение: 240000
AIOHTTP + FastAPI + PostgreSQL | база: FastAPI + PostgreSQL | вакансий: 8 | базовая медиана: 180000 | медиана: 420000 | изменение: 240000
Asyncio + FastAPI + PostgreSQL | база: FastAPI + PostgreSQL | вакансий: 6 | базовая медиана: 180000 | медиана: 420000 | изменение: 240000
Cassandra + FastAPI + PostgreSQL | база: FastAPI + PostgreSQL | вакансий: 6 | базовая медиана: 180000 | медиана: 420000 | изменение: 240000

ПОНИЖАЮТ МЕДИАНУ:
Docker + JavaScript + REST API | база: Docker + REST API | вакансий: 8 | базовая медиана: 250000 | медиана: 70000 | изменение: -180000
API + Git + Nginx | база: Git + Nginx | вакансий: 6 | базовая медиана: 250000 | медиана: 75000 | изменение: -175000
Git + JavaScript + Nginx | база: Git + Nginx | вакансий: 5 | базовая медиана: 250000 | медиана: 80000 | изменение: -170000
Docker + Docker-compose + SQL | база: Docker + SQL | вакансий: 5 | базовая медиана: 216250 | медиана: 50000 | изменение: -166250
Docker + REST + REST API | база: Docker + REST API | вакансий: 9 | базовая медиана: 250000 | медиана: 85000 | изменение: -165000
Git + MySQL + REST API | база: Git + REST API | вакансий: 7 | базовая медиана: 250000 | медиана: 90000 | изменение: -160000
GitHub + Nginx + SQL | база: Nginx + SQL | вакансий: 6 | базовая медиана: 250000 | медиана: 90000 | изменение: -160000
Django + GitHub + Nginx | база: Django + Nginx | вакансий: 6 | базовая медиана: 250000 | медиана: 91250 | изменение: -158750
Django + Linux + Nginx | база: Django + Nginx | вакансий: 9 | базовая медиана: 250000 | медиана: 95000 | изменение: -155000
Backend + Django + Nginx | база: Django + Nginx | вакансий: 6 | базовая медиана: 250000 | медиана: 96250 | изменение: -153750

Результаты сохранены.
```

Наибольшие положительные разницы в этой выборке встречаются у сочетаний с AIOHTTP и Cassandra, отрицательные — в отдельных сочетаниях с JavaScript, REST API и Nginx. Однако большинство крайних результатов получено по 5–9 вакансиям. Поэтому эти числа нельзя считать доказательством того, что добавление навыка само по себе повышает или снижает зарплату. На результат могут влиять специализация, уровень опыта и особенности вакансий. Дополнительно стоит учитывать, что признак `gross` не использовался для приведения зарплат к одному типу до/после налога.

6. Сравнение трёх регионов

Для анализа выбрала **Москву, Санкт-Петербург и Екатеринбург** — три региона с наибольшим количеством подходящих вакансий.

6.1. Количество вакансий и медианная зарплата

Регион	Python-вакансии	Из них с зарплатой в рублях	Медианная зарплата
Москва	1 910	500	**200 000 ₽**
Санкт-Петербург	525	186	**150 000 ₽**
Екатеринбург	152	63	**150 000 ₽**

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
[(.venv) > $ python region_salaries.py
РЕГИОНАЛЬНЫЙ АНАЛИЗ ЗАРПЛАТ
Москва | вакансий: 1910 | с зарплатой: 500 | медиана: 200000.0 руб.
Санкт-Петербург | вакансий: 525 | с зарплатой: 186 | медиана: 150000.0 руб.
Екатеринбург | вакансий: 152 | с зарплатой: 63 | медиана: 150000.0 руб.

Сохранено: region_salaries.csv
```

В Москве больше всего подходящих вакансий. Её медианная зарплата на **50 000 ₽** выше, примерно на 33%, чем в Санкт-Петербурге и Екатеринбурге.
Зарплата в рублях указана у 500 из 1 910 московских вакансий, 186 из 525 петербургских и 63 из 152 екатеринбургских, поэтому сравнение относится только к объявлениям с указанной зарплатой.

6.2. Востребованность навыков

Во второй таблице привела наиболее частые одиночные навыки, пары и тройки по каждому региону. Для расчёта использовала `area` и то же условие отбора Python-вакансий. Фрагмент расчёта:

```
region = (row["area"] or "").strip()
if region in REGIONS and role == "Программист, разработчик" and "python" in
name:
    skills = sorted({s.strip() for s in (row["key_skills"] or "").split(";")
if s.strip()})
    for size in (1, 2, 3):
        counts[region][size].update(combinations(skills, size))
```

В скобках в таблице указано количество вакансий.

Регион	Самый частый навык	Самая частая пара	Самая частая тройка
Москва	Python (1 361)	PostgreSQL + Python (615)	Docker + PostgreSQL + Python (223)
Санкт-Петербург	Python (391)	PostgreSQL + Python (190)	Docker + PostgreSQL + Python (69); Git + PostgreSQL + Python (69)
Екатеринбург	Python (114)	PostgreSQL + Python (55)	Git + PostgreSQL + Python (22)

```
РЕГИОН: Москва

Топ-5 комбинаций из 1 навыков:
Python 1361
PostgreSQL 641
Docker 435
Git 396
SQL 384

Топ-5 комбинаций из 2 навыков:
PostgreSQL + Python 615
Docker + Python 402
Python + SQL 374
Git + Python 371
Linux + Python 337

Топ-5 комбинаций из 3 навыков:
Docker + PostgreSQL + Python 223
Git + PostgreSQL + Python 189
Git + Linux + Python 177
Django Framework + PostgreSQL + Python 162
PostgreSQL + Python + SQL 146
```

```
[РЕГИОН: Санкт-Петербург

 Топ-5 комбинаций из 1 навыков:
[Python 391
 PostgreSQL 193
 Git 130
 Docker 106
 Linux 103

 Топ-5 комбинаций из 2 навыков:
 PostgreSQL + Python 190
 Git + Python 125
 Docker + Python 103
 Linux + Python 99
 Python + SQL 91

 Топ-5 комбинаций из 3 навыков:
 Docker + PostgreSQL + Python 69
 Git + PostgreSQL + Python 69
 Django Framework + PostgreSQL + Python 66
 PostgreSQL + Python + Redis 60
 Git + Linux + Python 50
```

```
РЕГИОН: Екатеринбург

Топ-5 комбинаций из 1 навыков:
Python 114
PostgreSQL 57
Git 43
Django Framework 30
Linux 28

Топ-5 комбинаций из 2 навыков:
PostgreSQL + Python 55
Git + Python 39
Django Framework + Python 30
Linux + Python 26
Python + SQL 25

Топ-5 комбинаций из 3 навыков:
Git + PostgreSQL + Python 22
Django Framework + PostgreSQL + Python 19
PostgreSQL + Python + SQL 16
Docker + PostgreSQL + Python 15
Git + Linux + Python 12

Сохранено: region_skills.csv
```

Во всех трёх регионах наиболее востребованная пара — **PostgreSQL + Python**.
Она встречается примерно в 32% подходящих вакансий Москвы и 36% вакансий Санкт-Петербурга и Екатеринбурга. В Москве чаще встречается тройка с Docker, а в Екатеринбурге — с Git. На мой взгляд, это показывает общую востребованность Python и PostgreSQL при небольших различиях в других технологических сочетаниях. Абсолютные частоты в Москве больше ещё и потому, что в выборке больше московских вакансий.

7. Профилирование времени и памяти

Для сравнения запустила каждую программу по **три раза** на одном датасете и с одинаковой фильтрацией вакансий. Программа `benchmark_run.py` измеряла длительность запуска и максимальное наблюдавшееся значение RSS-памяти процесса с

помощью `psutil`. Время включает подключение к RustFS, обработку и работу с результатами.

Подход	Запуск 1	Запуск 2	Запуск 3	Среднее время	Средний пик памяти
MapReduce, время	6,930 с	7,290 с	7,906 с	**7,375 с**	—
MapReduce, память	117,61 MiB	117,45 MiB	118,30 MiB	—	**117,79 MiB**
DuckDB, время	6,271 с	4,969 с	8,108 с	**6,449 с**	—
DuckDB, память	440,39 MiB	435,30 MiB	436,81 MiB	—	**437,50 MiB**

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
(.venv) > $ python benchmark_run.py
Введите секретный ключ RustFS:

mapreduce.py: запуск 1/3
Введите секретный ключ RustFS:
Время: 6.930 с
Пик памяти: 117.61 MiB

mapreduce.py: запуск 2/3
Введите секретный ключ RustFS:
Время: 7.290 с
Пик памяти: 117.45 MiB

mapreduce.py: запуск 3/3
Введите секретный ключ RustFS:
Время: 7.906 с
Пик памяти: 118.30 MiB

duckdb_skills.py: запуск 1/3
Введите секретный ключ RustFS:
Время: 6.271 с
Пик памяти: 440.39 MiB

duckdb_skills.py: запуск 2/3
Введите секретный ключ RustFS:
Время: 4.969 с
Пик памяти: 435.30 MiB

duckdb_skills.py: запуск 3/3
Введите секретный ключ RustFS:
Время: 8.108 с
Пик памяти: 436.81 MiB

Сохранено: benchmark_results.csv
```

На этих запусках DuckDB оказался **в среднем на 12,6% быстрее**, но использовал **примерно в 3,7 раза больше памяти**. Собственный MapReduce читает данные

последовательно и сворачивает результаты по чанкам, поэтому расход памяти оказался ниже. Время DuckDB немного колебалось между запусками; для более общего вывода понадобилось бы больше повторов и несколько объёмов данных. Измеренный пик — максимум, который удалось наблюдать при периодическом опросе процесса, а не гарантированный абсолютный пик за всё время.

Проверка отказа маппера

Отдельно проверила тестовый отказ на третьем чанке. Программа обнаружила ошибку после двух успешно обработанных чанков и не сохранила неполные результаты как итоговые. Повторный запуск отказавшего маппера в этом тесте не реализован: проверялось корректное прекращение расчёта.

```
macbook@MacBook-Pro-Macbook-9 ~/mapreduce-lab3
(.venv) > $ python test_mapper_failure.py
Введите секретный ключ RustFS:
Обнаружена ошибка: Тестовый отказ маппера
Обработано чанков до отказа: 2
Неполные результаты не сохранены
```

8. Вывод

В работе развернула RustFS, загрузила датасет вакансий и реализовала подсчёт отдельных навыков, пар и троек собственным MapReduce с чтением чанками из S3. Независимый подсчёт через DuckDB полностью совпал с полученными частотами.

На основе вакансий Python-разработчиков рассчитала медианные зарплаты для сочетаний навыков и сравнила Москву, Санкт-Петербург и Екатеринбург. Для данного датасета DuckDB в среднем работал немного быстрее, а собственная реализация MapReduce потребляла заметно меньше памяти. При интерпретации зарплатных разниц важно учитывать небольшой размер отдельных групп и ограничения исходных данных.